

# Color Recognition

This session will use OpenCV to perform red, green, blue recognition and display the recognition result through transmitted image.

Before operations, please prepare one object for each color: red, green, and blue.

## 1. Recognition Process

Firstly, obtain the RGB image from the camera. Scale and apply Gaussian blur to the image. Convert the image from RGB to Lab. (For detailed explanations regarding the Lab color space, you can refer to OpenCV Basic Lesson.)

Next, identify the object color in the circle by using color thresholds, followed by masking of the image portion. (Masking is used to hide certain part of the processed image).

After performing opening and closing operations on the object image, the object with the largest contour is circled.

Open operations: erosion followed by dilation. Remove small objects, smooth the contours of object and keep its area unchanged. It can eliminate small noise and break thin connection between objects.

Erosion: it can be used to remove small objects or features from an image, break thin connections between objects, and generally reduce the size of objects.

Dilation: it is useful for filling in gaps between objects, joining nearby objects, and generally increasing the size of objects.

Finally, the recognition results are displayed on the live camera feed.

## 2. Operation Steps

---

Note: the entered command should be case sensitive and “Tab” key can be used to complement the key words.

---

- 1) Start MentorPi and connect it to VNC.

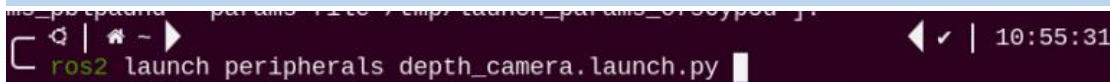
- 2) Click on  to open the command line terminal.
- 3) Input the command and press Enter to disable the app auto-start service.

```
~/stop_sh
```



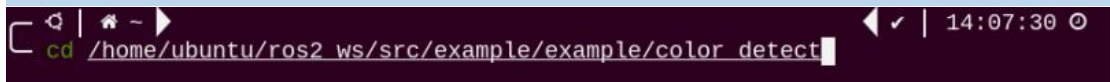
- 4) Run the following command to enable the camera node:

```
ros2 launch peripherals depth_camera.launch.py
```



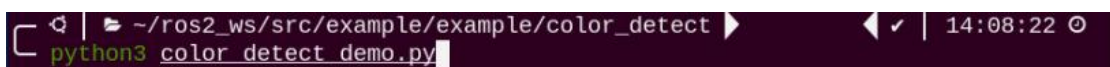
- 5) Open a new command line terminal. Enter the following command to navigate to the program directory.

```
cd /home/ubuntu/ros2_ws/src/example/example/color_detect
```



- 6) Enter the command. Press “Enter” to execute the program.

```
python3 color_detect_demo.py
```

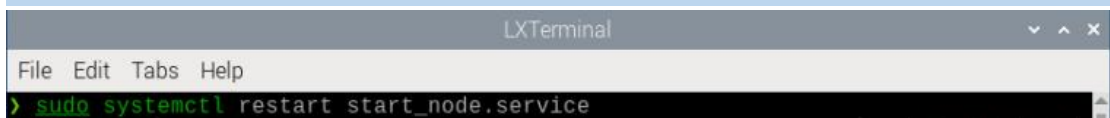


- 7) If you want to exit the game, press “Ctrl+C” in the terminal interface.
- After experiencing the game, you can enable the app service through commands or by restarting the robot. If the app is not enabled, the related app functions will not work. If the robot is restarted, the app will be automatically enabled.

Click  and enter the command. Press enter to start the app, and wait for the buzzer to beep.

Note: please enter the command in the system path, not in the Docker container.

```
sudo systemctl restart start_node.service
```



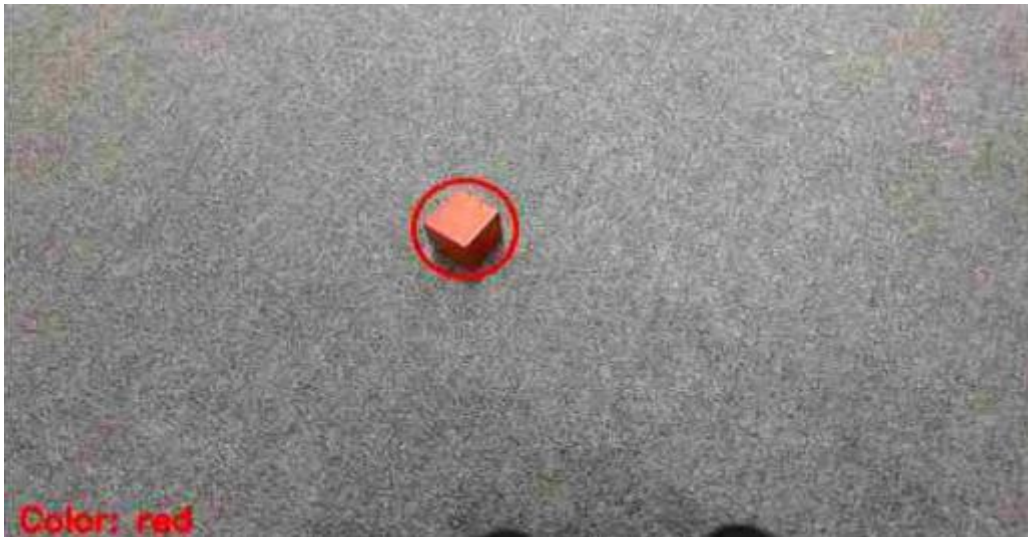
### 3. Program Outcome

---

Note: after the game starts, please ensure that there are no other objects containing recognized colors within the field of view of the camera to avoid affecting the implementation effect.

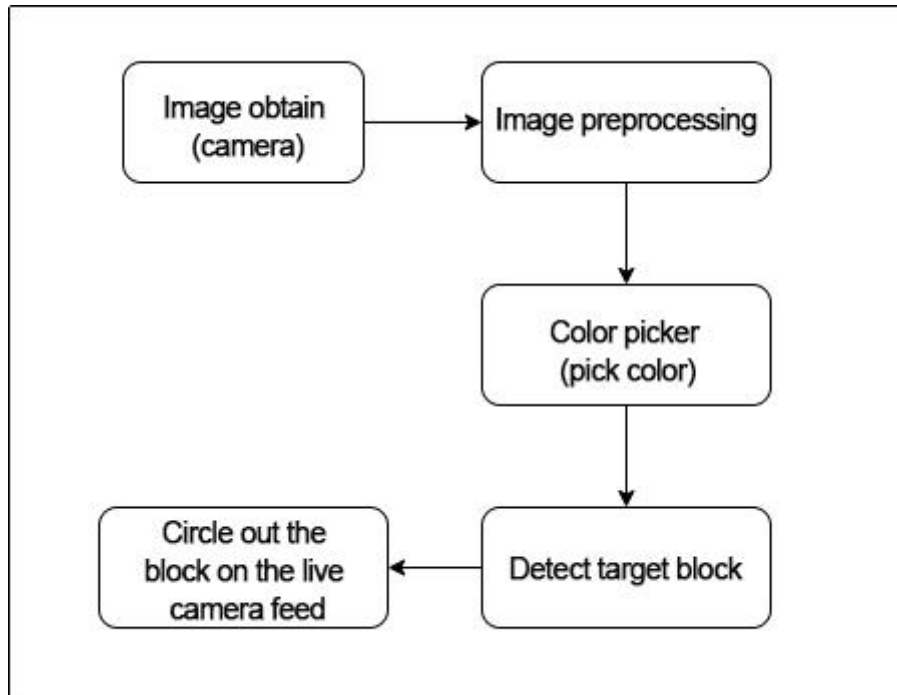
---

After the game starts, place the target object within the field of view of the camera. When the target object is recognized, it will be circled up in the same color and the color name will be printed in the lower-left corner of the screen. The program supports red, blue and green object recognition.



### 4. Program Analysis

The source code of the program is located in  
`/home/ubuntu/ros2_ws/src/example/example/color_detect/color_detect_demo.py`



## 4.1 Basic Configuration

### ◆ Read color threshold configuration file

Call the “get\_yaml\_data” function to obtain the color threshold configuration file.

```

37 lab_data = get_yaml_data("/home/ubuntu/software/lab_tool/lab_config.yaml")
38

```

### ◆ Obtain live camera feed

Create a subscription object to read the live camera feed

through the code “'/ascamera/camera\_publisher/rgb0/image', image\_callback, 1)”.

```

160 node.create_subscription(Image, '/ascamera/camera_publisher/rgb0/image',
    image_callback, 1)

```

## 4.2 Image Processing

The “run” function is called for image processing.

```

62 global color_list
63 global detect_color
64 lng_copy = lng.copy()
65 lng_h, lng_w = lng.shape[:2]
66
67 frame_resize = cv2.resize(lng_copy, size, interpolation=cv2.INTER_NEAREST)
68 frame_gb = cv2.GaussianBlur(frame_resize, (3, 3), 3)
69 frame_lab = cv2.cvtColor(frame_gb, cv2.COLOR_BGR2LAB) # 将图像转换到LAB空间(convert image to LAB space)
70
71 max_area = 0
72 color_area_max = None
73 areaMaxContour_max = 0
74
75 for i in ['red', 'green', 'blue']:
76     frame_mask = cv2.inRange(frame_lab, tuple(lab_data['lab']['Stereo'][i]['min']), tuple(lab_data['lab']['Stereo'][i]['max'])) # 对原图像和掩模进行位运算(perform bitwise operation on the original
77     # Image and the mask)
78     eroded = cv2.erode(frame_mask, cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))) # 腐蚀(erode)
79     dilated = cv2.dilate(eroded, cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))) # 膨胀(dilate)
80     contours = cv2.findContours(dilated, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)[-2] # 找出轮廓(find contours)
81     areaMaxContour, area_max = getAreaMaxContour(contours) # 找出最大轮廓(Find the largest contour)
82     if areaMaxContour is not None:
83         if area_max > max_area: # 找最大面积(Find the maximum area)
84             max_area = area_max
85             color_area_max = i
86             areaMaxContour_max = areaMaxContour
87     if max_area > 200: # 寻找最大面积(the maximum area is found)
88         ((centerX, centerY), radius) = cv2.minEnclosingCircle(areaMaxContour_max) # 获取最小外接圆(Obtain the minimum circumscribed circle)
89         centerX = int(common_val_map(centerX, 0, size[0], 0, lng_w))
90         centerY = int(common_val_map(centerY, 0, size[1], 0, lng_h))
91         radius = int(common_val_map(radius, 0, size[0], 0, lng_w))
92         cv2.circle(lng, (centerX, centerY), radius, range_rgb(color_area_max), 2) # 画圆(draw circle)

```

## ◆ Gaussian blur

Use the “cv2.GaussianBlur()” function from the cv2 library to apply Gaussian filtering to the image to remove noise.

```

69 frame_gb = cv2.GaussianBlur(frame_resize, (3, 3), 3)

```

The first parameter “frame\_resize” is the input image.

The second parameter “(3, 3)” is the size of the Gaussian convolution kernel.

The height and width of the kernel must be positive and odd numbers.

The third parameter “3” is the standard deviation of the Gaussian kernel in the horizontal direction.

## ◆ Color space conversion

Convert the image from BGR to LAB color space.

```

70 frame_lab = cv2.cvtColor(frame_gb, cv2.COLOR_BGR2LAB) # 将图像转换到LAB空间(convert image to LAB space)

```

## ◆ Color detection, erosion and dilation

Perform erosion and dilation operations on the binary image to smooth the contour edges in the image and facilitate the subsequent search for target contours.

```

77 frame_mask = cv2.inRange(frame_lab, tuple(lab_data['lab']['Stereo'][i]['min']), tuple(lab_data['lab']['Stereo'][i]['max'])) # 对原图像和掩模进行位运算(perform bitwise operation on the original
78 # Image and the mask)
79 eroded = cv2.erode(frame_mask, cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))) # 腐蚀(erode)
80 dilated = cv2.dilate(eroded, cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))) # 膨胀(dilate)

```

“frame\_mask”: the input binary image mask.

“cv2.getStructuringElement(cv2.MORPH\_RECT, (3, 3))”: generates a 3x3 rectangular structural element (kernel) that specifies the shape and size of the erosion operation. “cv2.MORPH\_RECT” specifies the rectangular shape. “eroded”: The binary image mask after erosion.

“cv2.getStructuringElement(cv2.MORPH\_RECT, (3, 3))”: uses the same structural element as erosion for the dilation operation.

### ◆ Get contour with the maximum area

Call the “cv2.findContours()” function from the cv2 library to find the maximum contour of the target color in the image.

```

81     areaMaxContour, area_max = getAreaMaxContour(contours) #找出最大轮廓(find the largest contour)
82     if areaMaxContour is not None:
83         if area_max > max_area: #找最大面积(find the maximum area)
84             max_area = area_max
85             color_area_max = i
86             areaMaxContour_max = areaMaxContour

```

Initialization: “contour\_area\_temp” and “contour\_area\_max” store the current contour area and the maximum contour area. “area\_max\_contour” stores the maximum contour.

Traverse Contours: Traverse each contour c in the contours list.

Calculate Area: Calculate the area of the contour c using “cv2.contourArea(c)”.

Determine Maximum Area: If the current contour area is greater than the known maximum area and greater than the threshold of 50, update the maximum area and maximum contour.

Return Value: The function returns the contour with the maximum area “area\_max\_contour” and its corresponding maximum area “contour\_area\_max”.

## 4.3 Feedback Action

### ◆ Circle the target object

Call the “drawContours()” function to draw a bounding box around the target object in the live camera feed.

```

124     cv2.putText(img, "Color: " + detect_color, (10, img.shape[0] - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.65, draw_color, 2)
125

```

The meaning of the parameters in the function parentheses is as follows:

img: The target image on which to draw the text.

"Color: " + detect\_color: The text string to be drawn, displaying the detected color.

(10, img.shape[0] - 10): The starting position of the text, in pixels, representing the coordinate of the lower left corner.

“10” is the x-coordinate of the text, starting from the left edge.

img.shape[0] - 10 is the y-coordinate of the text, indicating the position above

the bottom of the image.

`cv2.FONT_HERSHEY_SIMPLEX`: The font type.

0.65: The size of the text.

`draw_color`: The color of the text, represented by a BGR format tuple, such as (255, 0, 0) for blue.

The thickness of the text line, representing the pixel width of the border.